

Kafka 叢集版本定義

從 IBP 到 KIP-1170: Metadata 引導機制的統一演進

核心背景：什麼是版本定義？

版本不只是二進位檔的號碼，它是叢集能力的「共識狀態」。

版本控制涉及的 維度範圍



RPC 協定版本

MV 驅動

決定傳輸紀錄的 Schema。例如：是否支援新的標記欄位。由當前叢集活化的 `metadata.version` 決定開關。



功能開關 (Feature Gates)

MV 驅動

邏輯層面的解鎖。如：新的 Consumer Group 協定、分層存儲 (Tiered Storage) 的底層邏輯觸發。

參數與邏輯檢查

Binary 驅動

關鍵！ 檢查邏輯依賴代碼。即使 MV 為 4.3，若使用 4.4 Binary 啟動，系統會套用 4.4 的嚴格檢查規則，可能導致舊參數失效。



格式化佈局 (Format)

Binary 驅動

磁碟檔案結構。使用 4.4 工具執行 format，即便指定 MV=4.3，實體佈局仍可能包含 4.4 專屬屬性，導致無法降級回 4.3 啟動。

核心觀念：二進位代碼版本 (Binary Version) 是「上限與規則」，元數據版本 (MV) 是「當前啟用範圍」。

實際應用場景：什麼時候要注意？

叢集升級

更新 Binary 後，MV 尚未提升前。此時 Binary 的嚴格檢查已生效，但新特性尚未解鎖。

災難恢復

重刷硬碟格式化時。必須確保使用的 kafka-storage.sh 工具版本與既有叢集節點一致。

新節點加入

擴容時，如果新節點使用太新的版本格式化，可能會造成元數據同步後的解析異常。

版本定義的技術演進



過去 (Zookeeper)

依賴 `inter.broker.protocol.version`。配置分散在各個伺服器的 `server.properties`，升級時需手動滾動重啟並確保一致性。



現在 (KRaft)

引入 **`metadata.version`**。版本狀態被當作特殊的「Metadata 紀錄」存儲在 Raft 日誌中，由 Controller 統一管理。支援動態熱升級。

如何定義版本？（格式化階段）

⌘ 指令範例：

```
kafka-storage.sh (4.4) format --metadata-version 3.9
```

...

- ✔ **靜態寫入：**格式化時會將版本資訊寫入磁碟，作為「啟動引導」的依據。
- ⚠ **影響範圍：**這決定了叢集在尚未同步元數據紀錄前，所能理解的紀錄格式。

KIP-1170: 統一引導機制



痛點: 過去 KRaft 將啟動資訊分散在兩個檔案:



bootstrap.checkpoint

存儲叢集 Metadata 版本
(metadata.version) 與功能狀
態。



Zero Checkpoint

存儲 KRaft 投票成員 (Voter
Set) 與 Raft 層級的版本。



KIP-1170 解決方案

將兩者合併！所有引導紀錄統
一寫入 **零號快照 (Zero
Checkpoint)**。

實務案例分析：KAFKA-19192 引發的升級崩潰



當新版 Binary 遇上舊版 Metadata 檔案

在升級至 Kafka 4.0.0 的過程中，許多叢集遭遇致命的啟動失敗。

🔍 崩潰原因與核心印證

- ▶ 殘留檔案觸發嚴格檢查：伺服器啟動時讀取到磁碟上遺留的舊版 bootstrap.metadata 檔案。
- ▶ Binary 強制力：新版伺服器代碼要求該檔案內的 metadata.version 必須 $\geq 3.3-IV3$ 。若低於此版本，伺服器直接拒絕啟動。這完美印證了「檢查邏輯由 Binary 絕對驅動」。
- ▶ 歷史包袱的代價：這些舊引導檔案在叢集建置後已無實質作用，卻因缺乏清理機制成為升級地雷，促使社群加速架構改革。

架構演進：KIP-1170 統一叢集引導

為解決啟動資訊分散與舊檔案殘留引發的問題 (如 KAFKA-19192)，KIP-1170 徹底重構了引導機制。



Pre-KIP-1170 (過去)

啟動資訊被迫拆分。bootstrap.checkpoint 負責存儲 Metadata 版本與功能狀態；而另一個 0000...00.checkpoint 則專門負責存儲 KRaft 的控制紀錄與投票成員。雙軌制增加了升級與維護的風險。

Post-KIP-1170 (現在與未來)

全面統一！所有的引導紀錄被集中寫入 **零號快照 (Zero Checkpoint)**。不再產生 bootstrap.checkpoint，且系統會在成功載入後自動刪除舊檔案，從根本上拔除歷史包袱與版本驗證衝突的隱患。

什麼時候需要定義版本？

- ★ **初次安裝**：定義整個叢集的特性基準線 (Baseline)。
- 🔥 **功能升級**：當需要啟用如「分層存儲」或「新的事務協議」時。
- ✂️ **節點擴容**：新節點格式化時必須與叢集當前運行的版本相容，否則無法加入 Quorum。



提示：在 KIP-1170 之後，管理員不再需要擔心遺留的 bootstrap.checkpoint 導致啟動迴圈問題。

KIP-1170: 統一引導機制的影響

啟動時的版本解析流程

- **讀取 Zero Checkpoint**: 取得 metadata.version 與 KRaft 成員。
- **二進位相容性檢查**: 檢查本地 Binary 是否能處理該 MV 紀錄。
- **建立 Metadata Log**: 若為新格式化節點, 依賴 Binary 的格式化邏輯建立初始快照。

實務情境:

如果您使用 4.4 的程式碼進行 `--metadata-version 4.3` 的格式化:

- 程式碼會檢查 4.4 規範的參數。
- 磁碟會寫入 4.4 工具生成的 Zero Checkpoint。
- **結果**: 拿 4.3 的程式碼去啟動這個目錄時, 可能會因為無法識別 4.4 工具生成的特定 Checkpoint 欄位而失敗。

運行中定義：動態管理

Describe

kafka-features.sh describe

查看當前叢集的 finalized 版本。

Upgrade

kafka-features.sh upgrade

在運行中推升版本紀錄，觸發特性啟用。

Downgrade

注意！

並非所有版本都支援降級，升級前務必確認文件說明。

維運最佳實踐

- 🛡️ **升級順序**: 先更新所有 Broker 的二進位程式碼, 觀察穩定後, 再透過指令升級 Metadata 版本。
- 📁 **檔案檢查**: 在升級到支援 KIP-1170 的版本後, 觀察 bootstrap.checkpoint 是否已被系統自動清理。
- 💾 **備份**: 任何版本定義的變更都應在升級前備份元數據日誌目錄。

總結：邁向更穩健的 Kafka

透過 **KIP-1170** 的統一，版本定義從分散的配置文件轉變為單一的引導快照。

這不僅降低了維運的複雜度，也為未來更複雜的 Metadata 管理奠定了基礎。



掌握版本細節，守護叢集穩定

